

Relatório — Trabalho 3: Compilador Front-end

NeyScript

Disciplina: Aspectos de Compiladores **Aluno:** João Vitor **Tema da linguagem:** 100% voltada para **Neymar Jr.** **Ferramentas:** JFlex 1.9.1 (léxico) + CUP 11b (sintático/semântico) + Java 23
Repositório: <https://github.com/joaoviitorsx/neyscript>

1. Introdução

O objetivo do trabalho é construir o *front-end* de um compilador (analisador léxico, sintático e semântico) usando JFlex e CUP, e integrá-los em uma ferramenta única que processa código-fonte como entrada.

A linguagem criada — **NeyScript** — substitui as palavras-chave clássicas por termos do universo do Neymar Jr.: tipos primitivos viram estatísticas (`camisa10`, `ousadia`, `torcida`, `semGol`), funções viram **partidas**, condicionais viram **dribles** e **carrinhos**, repetições viram **prorrogações**, a impressão na tela é a `**narra**ção` e a função principal é o próprio `neymar`. Chamadas de função são feitas com o prefixo `assistencia`, simbolizando o passe que aciona a jogada.

2. Especificação da linguagem (tokens)

2.1 Tabela canônica de tokens

Token	Lexema	Equivalente	Justificativa temática
NEYNAR	<code>neymar</code>	<code>main</code>	Representa o personagem central do programa, como a função principal
CAMISA10	<code>camisa10</code>	<code>int</code>	Tipo inteiro; associado ao número clássico de craques
OUSADIA	<code>ousadia</code>	<code>float</code>	Tipo real; representa valores de precisão, habilidade, nota ou estatística

Token	Lexema	Equivalente	Justificativa temática
TORCIDA	torcida	string	Tipo texto; a "torcida" canta frases (cadeias de caracteres)
SEM_GOL	semGol	void	Função sem retorno, ou seja, sem "gol" produzido
PARTIDA	partida	declaração de função	Uma função vira uma "partida" executável
NARRA	narra	print	O narrador anuncia/imprime um valor na tela
DRIBLE	drible	if	Condição: se o drible passar, executa o bloco
CARRINHO	carrinho	else	Caminho alternativo quando a jogada é interrompida
PRORROGACAO	prorrogacao	while	Repetição enquanto o jogo continua
GOL	gol	return	Resultado final devolvido pela função
ASSISTENCIA	assistencia	chamada de função	Chamada de outra jogada/função
ID	gols, placar, dribles	identificador	Nomes definidos pelo programador
NUM_INT	10, 2, 7	inteiro	Literais inteiros
NUM_FLOAT	9.5, 10.0	real	Literais decimais
LIT_TEXTO	"Neymar Jr"	string literal	Literais de texto entre aspas

2.2 Operadores e delimitadores auxiliares

Categoria	Símbolos
Aritmético	+ - * /
Relacional	== != < > <= >=

Atribuição `=`

Delimitador `; , ( ) { }`

2.3 Comentários

- Linha única: `//` até o fim da linha
- Bloco: `/*` qualquer coisa, multi-linha `*/`

2.4 Exemplo mínimo

```
partida camisa10 soma(camisa10 a, camisa10 b) {
    gol a + b;
}

neymar() {
    torcida nome = "Neymar Jr";
    camisa10 total = assistencia soma(118, 136);
    narra(nome);
    drible (total >= 250) {
        narra("LENDA!");
    } carrinho {
        narra("ainda nao");
    }
}
```

3. Analisador Léxico (JFlex) — `src/lexer.flex`

A especificação JFlex define macros, regras de reconhecimento e ações Java. Cada palavra-chave é reconhecida **antes** do padrão `{Identifier}` para que identificadores não "absorvam" as *keywords* (regra de prioridade do JFlex). Literais retornam o valor já convertido para Java (`Integer`, `Double`, `String`). Comentários e *whitespace* são descartados; qualquer outro caractere dispara um erro léxico com posição.

Trecho representativo de `src/lexer.flex`:

```
/* macros */
Digit          = [0-9]
```

```

Letter          = [a-zA-Z_]
Identifier       = {Letter}({Letter}|{Digit})*
IntegerLiteral  = {Digit}+
FloatLiteral     = {Digit}+ "." {Digit}+
StringLiteral   = \"([^\\"\\n]|\\.)*\"
CommentSingle   = "//" [^\r\n]*
CommentMulti    = "/*" ~"*/"

%%

<YYINITIAL> {
    /* palavras-chave (antes de {Identifier}) */
    "neymar"      { return symbol(sym.NEYMAR);      } /* main */
    "camisa10"    { return symbol(sym.CAMISA10);    } /* int */
    "ousadia"     { return symbol(sym.OUSADIA);     } /* float */
    "torcida"     { return symbol(sym.TORCIDA);     } /* string */
    "drible"      { return symbol(sym.DRIBLE);     } /* if */
    "narra"       { return symbol(sym.NARRA);       } /* print */

    /* literais */
    {FloatLiteral} { return symbol(sym.NUM_FLOAT, Double.parseDouble(yytext())); }
    {IntegerLiteral} { return symbol(sym.NUM_INT, Integer.parseInt(yytext())); }
    {StringLiteral} { String s = yytext();
                        return symbol(sym.LIT_TEXT0, s.substring(1, s.length()-1)); }

    {Identifier} { return symbol(sym.ID, yytext()); }
    {Comment}    { /* descarta */ }
    {WhiteSpace} { /* descarta */ }
}
[^] { throw new RuntimeException("ERRO LEXICO: ..."); }

```

A classe gerada é `Lexer` (`gen/Lexer.java`), com método `next_token()` invocado pelo parser.

4. Analisador Sintático e Semântico (CUP) — `src/parser.cup`

4.1 Gramática (resumo)

```

programa      ::= lista_decl
lista_decl    ::= lista_decl decl | decl
decl          ::= decl_func | decl_neymar

decl_func     ::= partida tipo ID ( params ) bloco
decl_neymar   ::= neymar ( ) bloco

tipo          ::= camisa10 | ousadia | torcida | semGol

```

```

params      ::= param ( , param)* | ε
param       ::= tipo ID

bloco       ::= { stmts }
stmt        ::= decl_var | atrib | drible | prorroga | retorno | call_stmt | narra |
bloco

decl_var    ::= tipo ID ; | tipo ID = expr ;
atrib       ::= ID = expr ;
drible      ::= drible ( expr ) bloco [carrinho bloco]
prorroga    ::= prorrogacao ( expr ) bloco
retorno     ::= gol [expr] ;
call_stmt   ::= assistencia ID ( args ) ;
narra       ::= narra ( expr ) ;

expr        ::= expr op expr | - expr | ( expr )
              | assistencia ID ( args ) | ID | NUM_INT | NUM_FLOAT | LIT_TEXTO

```

Precedência (alta → baixa) segue C: -unário > * / > + - > comparações > == != > =. CUP reporta **0 conflitos** e gera gen/parser.java + gen/sym.java.

4.2 Análise semântica embutida

O bloco parser code { : ... : } contém:

- **TabelaSimbolos** — pilha de escopos (Deque<Map<String, Simbolo>>).
- **Simbolo** — guarda nome, tipo, se é partida, lista de tipos dos parâmetros e linha.
- **unificarTipos(t1, t2, op, linha)** — decide tipo resultante; concatenação de **torcida** com +.
- **compativelAtribuicao(destino, origem)** — permite **camisa10** → **ousadia**, rejeita demais.
- **checarArgs(simbolo, args, linha)** — valida **aridade e tipo de cada argumento**.
- **tipoRetornoAtual** — pilha com tipo de retorno da partida corrente para validar gol.
- **paramsAtual** — coleta tipos dos parâmetros na declaração alimentando **checarArgs**.

Exemplo de ação semântica:

```

tipo:t ID:nome ASSIGN expr:e SEMI
{:
    if (t.equals("semGol"))
        parser.erroSemantico("variavel '" + nome + "' nao pode ter tipo semGol",
tleft);
    if (!parser.tabela.declarar(new Simbolo(nome, t, false, null, tleft)))
        parser.erroSemantico("variavel '" + nome + "' ja declarada", tleft);
    if (!parser.compativelAtribuicao(t, e))
        parser.erroSemantico("atribuicao incompativel: '" + nome + "' (" + t + ") = " +

```

```
e, tleft);  
:}
```

5. Integração — `src/Main.java`

O driver `Main`:

1. Lê o arquivo fonte em UTF-8.
2. **Etapa 1 — Léxica:** instancia `Lexer`, percorre `next_token()` até EOF e imprime tabela `linha`
`· coluna · classe · lexema`.
3. **Etapa 2 — Sintática + Semântica:** passa novo `Lexer` ao `parser.parse()`. Erros acumulados em `parser.mensagensErro`.
4. **Etapa 3 — Relatório:** imprime tabela de símbolos e resultado: `G00000L!!` OU `CARTAO VERMELHO`.

Mapeamento código numérico → nome do token via **reflection** sobre `sym`, sem acoplamento manual.

6. Scripts de build/run

```
./build.sh    # JFlex -> CUP -> javac (gera gen/ e build/)  
./run.sh      # executa pipeline completo sobre input.txt  
./run.sh --lex arquivo.txt    # só analise lexica  
./run.sh --parse arquivo.txt  # lex + sint + sem (default)  
./run.sh input_erros.txt      # demonstracao de diagnosticos
```

7. Demonstração

7.1 Programa válido (`input.txt`)

Cobre todos os elementos pedidos:

- três partidas (retorno inteiro, flutuante e `semGol`),
- parâmetros tipados, declarações com e sem inicializador,
- `neymar()` como ponto de entrada,
- chamadas com `assistencia` (aridade e tipos conferidos),
- impressão com `narra`, laço `prorrogacao`, condicional `drible/carrinho`,

- operadores aritméticos e relacionais,
- literais `NUM_INT`, `NUM_FLOAT`, `LIT_TEXTO`,
- concatenação de torcida com `+`,
- coerção `camisa10` → `ousadia`,
- comentários de linha e de bloco.

Saída real da execução (`./run.sh`), resumida:

```
=====
ETAPA 1 - ANALISE LEXICA (Scanner JFlex)
=====
LINHA  COL   TOKEN                LEXEMA/VALOR
-----
14      1      PARTIDA
14      9      CAMISA10
14      18     ID                  somarGols
14      27     LPAREN
14      28     CAMISA10
14      37     ID                  temporadaA
...
(237 tokens no total)

=====
ETAPA 2 - ANALISE SINTATICA + SEMANTICA (CUP)
=====
>> programa analisado com sucesso.

=== TABELA DE SIMBOLOS (escopo global) ===
NOME           TIPO           CATEGORIA  LINHA  PARAMS
somarGols      camisa10      partida    14     [camisa10, camisa10]
mediaGols      ousadia       partida    20     [camisa10, camisa10]
anunciar       semGol        partida    31     [torcida]
neymar         camisa10      partida    38     []

=====
ETAPA 3 - RELATORIO FINAL
=====
G00000L!! programa aceito sem erros lex/sint/sem.
```

7.2 Programa com erros (`input_erros.txt`)

O compilador detecta **11 erros semânticos** sem abortar:

#	Linha	Erro
1	9	partida dobro já declarada

#	Linha	Erro
---	-------	------

2	14	partida <code>semGol</code> não pode retornar valor
3	19	variável <code>x</code> já declarada no escopo
4	22	atribuição incompatível <code>camisa10 = ousadia</code>
5	24	identificador <code>z</code> não declarado
6	26	variável não pode ter tipo <code>semGol</code>
7	29	operação <code>+</code> inválida entre <code>torcida</code> e <code>camisa10</code>
8	31	aridade errada: <code>dobro</code> espera 1 argumento, recebeu 3
9	34	tipo de argumento: esperado <code>camisa10</code> , recebeu <code>torcida</code>
10	36	partida <code>inexistente</code> não declarada
11	38	<code>x</code> não é partida

Saída real (`./run.sh input_erros.txt`), etapa final:

```
=====
ETAPA 3 - RELATORIO FINAL
=====
CARTAO VERMELHO - 11 erro(s) encontrado(s):
- ERRO SEMANTICO (linha 9): partida 'dobro' ja declarada
- ERRO SEMANTICO (linha 14): partida 'semGol' nao pode retornar valor
- ERRO SEMANTICO (linha 19): variavel 'x' ja declarada no escopo atual
- ERRO SEMANTICO (linha 22): atribuicao incompativel: 'y' (camisa10) = ousadia
- ERRO SEMANTICO (linha 24): variavel 'z' nao declarada
- ERRO SEMANTICO (linha 26): variavel 'w' nao pode ter tipo semGol
- ERRO SEMANTICO (linha 29): operacao '+' invalida entre tipos torcida e camisa10
- ERRO SEMANTICO (linha 31): partida 'dobro' espera 1 argumento(s), recebeu 3
- ERRO SEMANTICO (linha 34): argumento 1 de 'dobro': esperado camisa10, recebeu
torcida
- ERRO SEMANTICO (linha 36): partida 'inexistente' nao declarada
- ERRO SEMANTICO (linha 38): 'x' nao eh partida
```

8. Checagens semânticas implementadas

- Tabela de símbolos com escopos aninhados
 - Declaração antes do uso para variáveis e partidas
 - Detecção de redeclaração no mesmo escopo
 - Compatibilidade de tipos em atribuições (coerção `camisa10` → `ousadia`)
 - Operadores aritméticos exigem numéricos; `+` concatena `torcida`
 - Condições de `drible` e `prorrogaçao` devem ser numéricas
 - `assistencia` confere existência, categoria, **aridade e tipos dos argumentos**
 - Retorno (`gol`) compatível com tipo declarado da partida
 - Partidas `semGol` não retornam valor nem aparecem em expressões
 - `narra` aceita qualquer valor exceto `semGol`
-

9. Como reproduzir

```
./build.sh          # compila o front-end
./run.sh            # programa valido -> G00000L!!
./run.sh input_erros.txt  # demo diagnosticos -> CARTAO VERMELHO
./run.sh --lex input.txt  # so stream de tokens
```

Requisitos: JDK 11+ e bash. Jars em `lib/` (não precisam ser instalados).

10. Repositório

<https://github.com/joaoviitorsx/neyscript>

11. Apresentação em vídeo

Link: [inserir URL do YouTube aqui]